

LABORATORIO N° 03

Analizador Sintactico con FLEX y BISON

Objetivo

Desarrollar habilidades en la construcción de analizadores sintácticos utilizando Flex y Bison, comprendiendo la definición de Gramáticas Independientes del Contexto (GIC), la especificación de reglas gramaticales y la integración del analizador léxico con el analizador sintáctico para validar la estructura de un lenguaje.

Desarrollo

Un analizador sintáctico (parser) verifica que la secuencia de tokens producida por el analizador léxico siga las reglas gramaticales de un lenguaje. Bison es una herramienta que, a partir de una gramática especificada en notación BNF (Backus-Naur Form), genera automáticamente un analizador sintáctico LALR(1).

La integración Flex + Bison sigue el siguiente flujo:

```
Archivo fuente      --> [Flex: Analizador Léxico]      --> Tokens
                   --> [Bison: Analizador Sintáctico]  --> Árbol / Validación
```

Ejercicio 1: Analizador Sintáctico de Fracciones (N/N)

Construir un analizador léxico-sintáctico capaz de reconocer y validar fracciones de la forma N/N, donde N es un número entero positivo. El analizador debe indicar si la expresión ingresada es una fracción válida y rechazar entradas incorrectas.

Descripción

- Gramática Independiente de Contexto (GIC):
FRACCION --> NUMERO / NUMERO
NUMERO --> [1-9][0-9]* | 0
- Donde:
 - FRACCION es el símbolo inicial (axioma).
 - NUMERO es un token (símbolo terminal) que representa un entero no negativo.
 - '/' es el token de división (símbolo terminal literal).

Entregable

Un programa que lea una expresión desde la entrada estándar, valide que sea una fracción del tipo N/N e imprima un mensaje indicando si es válida o no, incluyendo detección del denominador cero.

Ejercicio 2: Analizador Sintáctico de Expresiones Aritméticas

Construir un analizador sintáctico para expresiones aritméticas que involucren suma, resta, multiplicación y división, respetando la precedencia y asociatividad de operadores, y con soporte para paréntesis.

Descripción

- **GIC:**

$$\begin{aligned} E &\rightarrow E + E \mid (E) \mid T \\ T &\rightarrow \text{NUMERO} \end{aligned}$$

Entregable

Un analizador que valide y evalúe expresiones aritméticas ingresadas por el usuario, respetando precedencia de operadores. Mostrar el resultado si la expresión es válida o un mensaje de error si no lo es.

Ejercicio 3: Analizador Sintáctico de Sentencias de Asignación

Construir un analizador léxico-sintáctico para validar sentencias de asignación de un lenguaje simple, donde una variable (identificador) recibe el valor de una expresión aritmética.

Descripción

- **GIC:**

$$\begin{aligned} \text{programa} &\rightarrow \text{sentencia ';' } \\ &\quad \mid \text{programa sentencia ';' } \\ \\ \text{sentencia} &\rightarrow \text{IDENTIFICADOR '=' expresion} \\ \\ \text{expresion} &\rightarrow \text{expresion '+' termino} \\ &\quad \mid \text{expresion '-' termino} \\ &\quad \mid \text{NUMERO} \end{aligned}$$

Entregable

Un programa que lea un bloque de sentencias de asignación (una por línea, terminadas en ';') y valide su estructura sintáctica, reportando cada asignación reconocida o los errores encontrados.